

List of Experiments (UNIT-1) UCS553

(WEEK 3)

Q1. Develop a Java application for an online food delivery company that manages different types of food orders. Create packages: food.model, food.service, food.utility, food.main. Create an abstract class: FoodOrder having: orderId, customerName, amount and static member: restaurantName. Include: abstract double calculateDeliveryCharge(); Create an interface: Discountable having double applyDiscount(); Create two subclasses: RegularOrder, PremiumOrder.

The delivery charges for Regular order is Rs. 80 and for Premium is Rs. 50. The discount for Regular orders is 10% and for Premium orders is 15%.

Use: private data members, getters/setters, static object counter, static method to display total orders, constructors, method overriding concepts.

Create: OrderUtility to validate amount, validate customer name and generate order summary.

In the driver(Main) program:

- i. create at least six orders,
- ii. store them using an array of FoodOrder
- iii. display bill
- iv. display discount
- v. display delivery charge
- vi. display final payable amount

Q2. A university wants to develop a Smart Library Resource Management System to manage Books, Journals, and Digital Resources.

Create the following package structure:

- library.model
- library.service
- library.util
- library.main

Create an abstract class LibraryResource with: resourceId, title, author, static libraryName, constructor, abstract method: double calculateFine(int overdueDays);

Create an interface: Printable having method void printDetails();

Create two classes: Book, DigitalResource that extend LibraryResource and implement Printable

The fine calculation rules are: Book Rs. 5/day and Digital Resource Rs. 2/day

Use: private variables, protected methods (where appropriate), public methods, static variable, static method to display total resources created.

Maintain a static object counter that counts every resource created.

Create a utility class: InputValidator inside the utility package that validates Resource ID (>0) and Fine days (≥0)

In the main package:

- i. create at least five objects,
- ii. store them in an array,
- iii. display complete details,
- iv. calculate total fine of all overdue resources.